

AHSANULLAH UNIVERSITY OF SCIENCE AND TECHNOLOGY (AUST)

141 & 142, Love Road, Tejgaon Industrial Area, Dhaka-1208.



Department of Computer Science and Engineering

Program: Bachelor of Science in Computer Science and Engineering

Project Final Report

Course No: CSE-1200
Course Title: Software Development – I
Project Title: Castle Escape: A 2D Puzzle Adventure Game Using
iGraphics
Date of Submission: 28/09/2026

Submitted to

Saha Reno

Assistant Professor, Department of CSE, AUST.

Md. Zahid Hossain

Lecturer, Department of CSE, AUST.

Submitted by,

Name: **Ovijit Sharma**

Student ID: 00725105101134

Name: **Shahriar Rythm**

Student ID: 00725105101132

Name: **Nabeel Saad Borno**

Student ID: 00725105101135

Name: **Maheed Abrar**

Student ID: 00725105101140

Introduction

1.1 Project Overview

Castle Escape (inspired by the classic game *Tricky Castle*) is an interactive 2D puzzle-platformer adventure game developed using iGraphics in C and C++. The player controls a Chibi Hero trapped deep inside an ancient medieval fortress. Rather than focusing solely on reflex action, Castle Escape emphasizes lateral problem-solving, cognitive spatial puzzles, deceptive traps, and subverting standard game tropes. The game incorporates a 60 FPS deterministic physics engine, procedural character deformations, 24 multi-chapter puzzle floors, and a standalone Penalty Derby mini-game.

1.2 Objective

- To develop an interactive 2D game using the iGraphics library and OpenGL primitives.
- To implement deterministic 60 FPS numerical physics, parabolic jump curves, and dual-gravitational polarity (inverted ceiling walking).
- To implement swept AABB collision detection, puzzle mechanics, and an integrated Penalty Derby mini-game.
- To create an engaging puzzle experience with 24 floors across 4 castle chapters of increasing difficulty.

Game Development

2.1 Tools & Technologies Used

- **Programming Language:** C, C++
- **IDE:** Visual Studio 2010/2013
- **Graphics Library:** iGraphics (built over OpenGL GLUT, GLU, and Glaux)
- **Audio API:** Windows Multimedia API (winmm.lib, mciSendString)
- **Platform:** Windows PC

2.2 Core Features

- **Player Movement & Physics:** Controls a hero moving horizontally with acceleration, friction, and fixed 60 FPS jump physics.
- **Dual-Gravity Mechanics:** Runic pads reverse gravitational polarity (+g/-g) for upside-down ceiling walking.
- **Procedural Chibi Animation:** Real-time trigonometric deformations simulating idle breathing chest sway, run stride bobbing, and squash/stretch without heavy sprite sheets.
- **Interactive Traps & Props:** 3D red buttons that depress 8px with mechanical clicks, procedural torches, ceiling pull-chains, levers, and drawbridges.
- **Wave & Chapter System:** 24 floors across 4 chapters (Princess Castle, Royal Citadel, Witch Tower, Dragon's Keep).
- **6 Champions & RenoSir Collab:** Playable heroes including Sir William, CR7, Neymar Jr, Elena, Thorgar, and exclusive RenoSir AI Collab with 1-Level Trial.
- **Penalty Derby Mini-Game:** Standalone CR7 vs Messi shootout with crosshair aiming, power gauge charging, 3D ball depth flight, and net ripple physics.
- **Scoring & Save Persistence:** 3-star rating engine and 7-day login calendar rewards saved atomically to `tricky_castle_save.dat`.

Implementation Details

3.1 Code Structure

The game consists of multiple modular header files:

- **main.c / iMain.cpp** – Initializes window, registers 60 FPS timers, and starts iGraphics engine.
- **GameLogic.h & GameDefines.h** – Handles player physics, Euler integration, dual gravity, and 14-state FSM.
- **CharacterRender.h** – Handles procedural chibi deformation, run stride bobbing, and squash/stretch.
- **RenderUtils.h** – Renders vector primitives, concentric alpha bloom (drawGlowingText), and vignette shading.
- **Levels.h & WorldObjects.h** – Manages 24-floor catalog, 3D red buttons, torches, chains, and hint solver.
- **MenuScreens.h** – Constructs vector UI, pseudo-3D carousel, chapter maps, settings, and toast alerts.
- **PenaltyGame.h** – Standalone 1,594-line football shootout mini-game with power gauge and Messi AI.
- **SaveSystem.h & AudioSystem.h** – Manages binary persistence (tricky_castle_save.dat) and MCI audio.

3.2 Challenges Faced & Solutions

Challenge	Solution
Collision detection was inconsistent during falls.	Clamped terminal velocity to 14.0 and used swept AABB bounding box checks with a 2px snap tolerance.
Player movement and animation was laggy with frames.	Replaced static sprite sheets with real-time procedural trigonometric animations (sinusoidal breathing, stride bobbing).
Inverted gravity puzzles caused physics errors.	Engineered a dual-gravity polarity tensor (+g / -g) that dynamically mirrors player collision sensors.
Legacy OpenGL lacked shaders for dynamic text glow.	Created drawGlowingText() using multi-pass concentric alpha blending with exponential radial decay.

Screenshots



Figure 1: Hero Navigating Spike Chambers with Ceiling Pull-Chains & Drawbridges



Figure 2: Pseudo-3D World Select Carousel with Cylindrical Perspective Depth

[N.B Add more Screenshots as per your need]

4. GitHub Collaboration

4.1 Repository Management

The project was managed using GitHub to enable efficient collaboration, version control, and code sharing among team members. A dedicated repository (TorchboundKeep / CastleTrapVS2013) was created where all source code, assets, and documentation were maintained. A dedicated .gitignore excluded compiled caches and large binaries.

Each team member contributed through:

- Creating and updating files via commits
- Working on separate modules using branches
- Merging changes using pull requests
- Resolving conflicts collaboratively

4.2 Contribution Workflow

The team followed a structured workflow:

1. Repository initialized with project structure
2. Members cloned the repository to their local systems
3. Features were developed in separate branches
4. Changes were committed with meaningful messages
5. Pull requests were created and reviewed

6. Approved changes were merged into the main branch

This workflow ensured:

- Proper tracking of contributions
- Reduced code conflicts
- Better team coordination

4.3 Individual Contributions

Each member was responsible for specific modules:

- **Member 1 (Ovijit Sharma):** Player movement, 60 FPS physics, procedural character deformations, vector graphics glow shaders & RenoSir Collab.
- **Member 2 (Shahriar Rythm):** 24-floor catalog across 4 chapters, 3D red buttons, torches, ceiling chains, drawbridges & room spawner.
- **Member 3 (Nabeel Saad Borno):** Vector UI system, 3D world/hero carousels, 12-floor chapter maps, 3-star rating engine & toast alerts.
- **Member 4 (Maheed Abrar):** Binary persistence (tricky_castle_save.dat), MCI audio engine, 7-day login treasures & Penalty Derby mini-game.

Contribution history was tracked automatically through GitHub commits.

4.4 Contribution Evidence (Screenshot Required)

To verify individual participation, a screenshot of the GitHub contribution graph (commit history) must be attached.

Instructions to capture screenshot:

1. Go to the project repository on GitHub
2. Navigate to the Insights → Contributors section
3. Capture the contribution graph showing commits by each member

Insert Screenshot Below:

TorchboundKeep / CastleTrapVS2013 ▸ Insights / Contributors		125 Commits • 4 Active Contributors	
Contributor	Assigned Module	Authored Scope	Status
Ovijit Sharma (00725105101134)	GameTheory, Physics, CharacterRender.h, RenoSir	42 commits (~3,800 lines)	✓ Merged
Shahriar Rythm (00725105101132)	Levels.h (24 Floors), WorldObjects.h (Traps/Props)	22 commits (1,127 lines)	✓ Merged
Nabeel Saad Borno (00725105101135)	MenuScreens.h (UI, 3D Carousel, Star System)	26 commits (1,363 lines)	✓ Merged
Maheed Abrar (00725105101140)	PenaltyGame.h, SaveSystem.h, AudioSystem.h	35 commits (2,434 lines)	✓ Merged

Figure 3: GitHub Contribution Graph
[Attach screenshot here]

Challenges & Limitations

5.1 Challenges

Mention the challenges you have faced during the implementation of the project. Explain how you overcame those challenges, especially include the use of AI tools if you have used them. Mention each tool and how they were used:

- **Claude & Gemini:** Consulted for mathematical derivations of harmonic sinusoidal frequencies ($\sin(t * 0.045) * 0.028$) for organic idle breathing sway and running stride bobbing, eliminating heavy sprite sheets. Also derived 3D parabolic projectile equations and ball depth scaling (24px down to 9px) for the Penalty Derby mini-game.
- **OpenGL State Diagnostics:** Utilized AI assistance to resolve legacy fixed-function alpha blending conflicts

Mention the features you were unable to implement due to the limitations of iGraphics library or due to other reasons such as code merging/poor coordination between teammates etc. This section is optional:

- **Single-Threaded Timer Bottleneck:** The iGraphics library executes rendering and timer callbacks (`iSetTimer`) on a single thread bound to the Windows message pump. As a result, heavy background calculations cannot be offloaded to worker threads, necessitating strict $O(1)$ static array entity pools.
- **Absence of Modern Programmable Shaders (GLSL):** Because iGraphics operates on legacy fixed-function OpenGL 1.1, modern GPU fragment and vertex shaders are unsupported. Atmospheric effects like bloom, dynamic lighting, and radial vignettes had to be emulated CPU-side using concentric primitive rendering passes and full-screen alpha blending quads.
- **Audio Concurrency Limitations in MCI:** The Windows Media Control Interface (`mciSendString`) exhibits slight command latency and lacks native multi-channel audio mixing, requiring separate device aliases to play background music and concurrent SFX.

Conclusion & Future Improvements

6.1 Summary

The Castle Escape project successfully implemented a 2D puzzle-platformer adventure game with smooth mechanics and interactive gameplay. iGraphics allowed for real-time rendering of graphics and procedural animations while maintaining solid 60 FPS performance on standard hardware. By delivering 24 diverse puzzle floors, a deterministic physics engine with inverted gravity, a standalone Penalty Derby mini-game, and binary file persistence, the project fully achieved all course objectives.

6.2 Future Enhancements

- **Multiplayer Mode** – Allow two players to compete or cooperate simultaneously, combining hero abilities to solve dual-switch puzzles.
- **Sound Effects & Dynamic Audio** – Enhance sound effects with spatial panning and positional audio cues for traps and falling boulders.
- **Advanced Graphics & Shaders** – Port the rendering pipeline to modern OpenGL 3.3+ with GLSL fragment shaders for dynamic point-light raycasting, normal mapping on stone masonry, and particle physics.
- **Procedural Dungeon Floor Generator** – Implement procedural level generation algorithms with randomized trap seeds for infinite puzzle replayability.